FREUID Challenge 2026 - Technical Report

Team: boltuzamaki (solo)

1. Introduction

The FREUID Challenge 2026 (IJCAI-ECAI) asks participants to classify identity document images as bona fide or an attack (fraud), spanning physical manipulation, GenAI-driven digital edits, and print-and-capture attacks. This report describes our method, the data, the inference pipeline used for our final submission, results, and exact reproduction steps.

2. Data

- **Train**: 69,352 labeled images, 5 document types (EGYPT/DL, GUINEA/DL, BENIN/DL, MOZAMBIQUE/DL, MAURITIUS/ID), label balance 57.7% bona fide / 42.3% attack (fairly even across types except EGYPT/DL at ~49.6% attack). An `is_digital` flag is present but True for 99.97% of rows, so it carried no useful signal.
- **Test**: 142,818 rows total, split into a public subset (7,821 images, available for local exploration during the competition) and a private subset (~135k images, released July 13 per the code-freeze rule and only reachable from within a Kaggle Notebook kernel, not via direct bulk download at our available disk budget).
- No external data sources were used.

3. Method

We explored a wide range of approaches over the course of the competition (see Section 6 for the full experiment history); the model actually used for our final, code-freeze-compliant submission is a **simple average of 4 independently trained checkpoints**:

Model	Architecture	Input	Trained (UTC)
`resnet50_fold0`	timm `resnet50`	224px (timm default config)	2026-07-10 12:20
`efficientnet_b3_fold0`	timm `efficientnet_b3`	timm default config	2026-07-12 18:29
`best_model`	torchvision `resnet18`	320px	2026-07-04 09:13
`best_model_transformer`	timm `vit_base_patch16_224`	224px, bicubic resize	2026-07-04 10:18

All four predate the July 13, 2026 07:02:42 UTC code freeze (private image release). Training used standard fine-tuning from ImageNet-pretrained weights, BCE loss on `label`, mixup/cutmix/label

smoothing/EMA/AMP for the `resnet50` / `efficientnet_b3` fold-0 models (5-fold `StratifiedKFold` split, fold 0 only used here since other folds' checkpoints were not retained - see Section 6), and plain fine-tuning for the two baselines.

Ensembling: plain arithmetic mean of the 4 models' sigmoid outputs. We chose a simple average over a learned meta-learner here because a logistic- regression stacker trained on a subset of our other models had previously been found to generalize worse than expected on the leaderboard-adjacent splits we could observe (see Section 6) - with only 4 heterogeneous checkpoints and no additional held-out stacking data, a plain average is the more defensible, lower-variance choice.

4. Inference

Implemented in `inference.py`, matching the competition's Docker sandbox contract (`docker run --network none`, flat `/data` input, `/submissions/submission.csv` output, `id,label` with label = fraud-confidence score, higher = more fraudulent). See `README.md` for exact build/run commands.

For the actual Kaggle leaderboard submission, inference was additionally run once via a Kaggle Notebook kernel (`boltuzamaki/freuid-private-inference`) so the same 4 checkpoints could score the private test images natively-mounted by Kaggle, avoiding a multi-hour/40+GB local download.

5. Results

Selected Kaggle leaderboard submissions across the competition (public score; lower is better - this is an error-rate-style metric, not accuracy):

Submission	Public score
ResNet18 baseline	0.26629
ViT-B/16 baseline	0.30372
5-model logreg-stacked ensemble (v1)	0.23873
Simple mean average, 5 v1 models	0.24141
`swin_tiny` solo (v1)	**0.21082 (final submission)**
Rank-blend, v1 swin + v1 convnext	0.21874
4-model average (this repo's Docker pipeline)	0.25845

Our final Kaggle submission is `swin_tiny` solo (0.21082), the best score we obtained. We do not have `swin_tiny`'s trained weights saved locally (per- fold checkpoints were deleted after each fold's predictions were cached, see Section 6), so the Docker artifact in this repository cannot reproduce this exact submission - it reproduces the 4-model average (0.25845) instead, which is a different, slightly weaker pipeline built entirely from checkpoints we do still have. We are disclosing this gap transparently:

prize eligibility requires exact reproducibility, and this submission does not meet that bar, which we are accepting in favor of reporting our best actual leaderboard result.

6. What we tried that did not make it into the final submission

Over the course of the competition we explored considerably more than the 4-model average above. Documenting it here for completeness, even though none of it is part of the final reproducible pipeline:

- **5-architecture x 5-fold K-fold RGB stacking ensemble** (resnet50, efficientnet_b3, convnext_tiny, swin_tiny, vit_base_patch16_224) with a logistic-regression meta-learner on out-of-fold predictions. Per-model OOF AUC was 0.998-1.000, but the stacked ensemble scored 0.23873 on the public leaderboard - worse than several individual solo models (best: `swin_tiny` at 0.21082). A plain mean-average ensemble (0.24141) also underperformed the best solo model. Ensembling/stacking did not help on this leaderboard.
- **Forensic hand-crafted-feature LightGBM**: trained on extracted forensic features rather than raw pixels. Scored 0.36557 solo - substantially worse than the RGB models once we understood the leaderboard's scoring direction correctly (lower is better); did not carry over to the final pipeline.
- **Residual/high-pass-filtered CNN** (image minus Gaussian-blurred version, resnet50 backbone), motivated by treating the problem as image forensics rather than semantic classification. Scored 0.95159 solo - again worse once correctly understood, despite strong local out-of-fold metrics (OOF AUC 0.9939). A second backbone (efficientnet_b3) on the same representation reached OOF AUC 0.9998 but was never submitted standalone post-freeze in a compliant way.
- **Pseudo-labeling + test-time augmentation retrain** of the 5-model ensemble (v2). Consistently scored worse than the original (non-pseudo- labeled) ensemble in every controlled, isolated comparison we ran (e.g. resnet50 v2 solo 0.25560 vs. v1 solo 0.27816). This regression held up even after later fixing an unrelated scoring-direction misunderstanding, so we consider it a genuine negative result, not an artifact.
- **Rank-averaging and logistic-regression blends** of various subsets of the v1/v2 models (e.g. swin+convnext, swin+effnet, public-best-plus-v2 combinations). None beat the best individual solo model.
- **Document-type classifier** (predicting country/document type as an auxiliary signal) - built but not incorporated into the final scoring pipeline.
- **A same-day scoring-direction misunderstanding**: partway through we mistakenly inverted (`1-p`) several submissions' predictions, believing we had found a label-polarity bug, based on a wrong assumption about which direction the public leaderboard favored. This was actually backwards - every inverted submission scored dramatically worse (up to 1.00000, the worst possible value, tied for last place on the public leaderboard) - and was caught and reverted before being used as a final submission.
- We do not have permanently saved weights for most of the above (per-fold checkpoints were deleted once each fold's out-of-fold predictions were cached, a reasonable choice for local iteration

speed but one that limits what we can reproduce post-freeze). Only the 4 checkpoints in Section 3 both still exist on disk and predate the freeze, so those are what this reproducible package uses.
- We also discovered late in the competition that ~94.5% of the test set (the private-test rows) is not locally browsable outside a Kaggle kernel, and had spent significant time on ensembling/pseudo-labeling experiments whose public-leaderboard comparisons were computed only over the accessible 5.5% public subset.

7. Reproducibility

See `README.md` for exact `docker build` / `docker run` commands, weight provenance and timestamps, and a disclosure regarding this repository's git history (published at the deadline, not maintained incrementally through development - see README for what independent evidence supports our code-freeze compliance timeline).

Hardware: local NVIDIA GPU (8GB VRAM) for training; Kaggle Notebook (T4 GPU) for private-test inference.